

CSL351 : COMPUTER NETWORKS
Assignment - 7

Name - Ashish Ranjan
ID no. - 12340370

Drive Link :

<https://drive.google.com/drive/folders/1HET5GMssYovWi4SMOpSpQP3wbDVcFHHh?usp=sharing>

It contains .cc code for the PART2 named as as7_que2.cc, shell script file to generate the output for all the questions as run.sh , gnuplot codes to generate graph for each TCP variants as cwnd.gnuplot , cwnd.gnuplot, in the results folder the .txt files used by gnuplot generated by the .cc code is present, the plots are present in a separate folder named plots, the Pcap files created by the .cc code is in the Pcap Folder.

Folder Structure :

Inside **Part1 folder** 4 pdfs which were used for the algorithms named as TCP{variant_name}.pdf

Inside **Part2 folder results** folder containing the .txt files used for the gnuplot ,**plots** folder for the output image of the gnuplots, [run.sh](#) for the shell script used in PART2 , and the .cc code as as7_que2.cc and the gnuplots codes as cwnd.gnuplot and rtt.gnuplot used in PART2

Report From Next Page

PART 1

TCP Cubic :

CUBIC which was modified from BIC-TCP , is a high speed TCP congestion control protocol that serves as the default TCP algorithm in Linux. Instead of the standard linear window growth, CUBIC modifies the window growth into a cubic function dependent on the real time elapsed since the last congestion event . This lets the algorithm update and function independent of change in RTT.

Algorithm Details :

- 1. Multiplicative Decrease:** Upon detecting a packet loss event, CUBIC records the window size where the loss occurred as the maximum (W_{max}) and decreases the congestion window by a multiplicative factor of β , which is set to 0.2.
- 2. Concave Region:** After recovering from the loss, the window increases following the concave profile of the cubic function. The window grows rapidly at first but slows down as it gets closer to W_{max} , allowing the protocol to avoid sudden bursts of packet losses.
- 3. Convex Region (Max Probing):** Once the congestion window passes the saturation point (W_{max}), it forms a plateau and then enters the convex growth profile. This phase slowly increases the window initially and gradually accelerates its growth rate for a new maximum, assuming bandwidth in the network may have increased.
- 4. TCP-Friendly Mode:** If CUBIC determines that its calculated window size is smaller than what standard TCP would achieve in the same elapsed time, it switches to a TCP mode and matches the standard TCP window size.
- 5. Fast Convergence Heuristic:** To help new flows gather bandwidth, CUBIC checks if the newly recorded W_{max} during a loss is less than the previous W_{max} . If so, it further reduces the recorded W_{max} , forcing the flow to hit its growth plateau earlier and release more bandwidth for competing flows to catch up.

6. **Timeout** : When a timeout occurs, CUBIC triggers a reset function that clears all of its internal state variables, including the previously recorded maximum window size and minimum delay. In Linux , if the protocol later realizes that the timeout was a false alarm, it utilizes an "undo" mechanism to revert the congestion window back to its original size recorded just before the drop occurred.

Suitable Scenarios where it works well:

1. **Fast and Long-Distance Networks (High BDP)**: CUBIC is highly effective in networks with a large bandwidth-delay product, where standard TCP suffers from severe under-utilization due to sluggish, RTT-dependent window growth.
2. **Environments with Competing Flows of Different RTTs**: Because its window growth rate is based on elapsed time rather than round trips, CUBIC achieves excellent RTT-fairness, allowing flows with different RTTs to achieve approximately equal window sizes when competing at the same bottleneck.
3. **Short RTT and Low-Speed Networks**: In networks where standard TCP already performs well, CUBIC successfully operates in its TCP-friendly region, scaling similarly to standard TCP without aggressively stealing bandwidth from competing standard TCP flows
4. **Intra Protocol Fairness**: CUBIC works highly effectively when multiple CUBIC flows compete for the same bottleneck, converging smoothly to an equal, fair share of the bandwidth within a short period without causing major network disturbances.

Suitable Scenarios where it fails :

- 1. Rapid Convergence Requirements:** CUBIC trades fast convergence for stability; by setting the multiplicative decrease factor (β) to a smaller value of 0.2, it suffers from a slower convergence speed compared to protocols that might use a more adaptive setting.
- 2. Sudden and Drastic Increases in Capacity:** Because the algorithm spends a long time at the plateau around its previous saturation point (W_{max}), it can be sluggish in discovering a new saturation point if the available bandwidth suddenly increases far beyond the previous maximum.
- 3. Low Statistical Multiplexing Environments:** In environments where only a very few flows are competing, CUBIC's convergence speed may be noticeably slow, though this is generally not an issue on the highly multiplexed public Internet

Delay Based - TCP Vegas

Vegas was designed to improve throughput and reduce packet loss. Vegas monitors changes in round trip times to react to network saturation. Vegas operates on the principle that the congestion window should only be reduced due to losses that happen at your current sending rate not because of losses that happened at an earlier higher sending rate unlike Reno.

Algorithm Details :

1. **Accurate RTT Calculation** : Vegas records the exact system clock time when a segment is sent and again when its ack arrives, allowing for a highly precise rtt estimate.
2. **Faster Retransmission** : When Vegas receives a dup ack, it checks if the time since the segment was sent exceeds the timeout value if true then it retransmits immediately without waiting for three duplicate ACKs.
3. **Expected vs. Actual Rate**: Per rtt Vegas calculates an expected throughput, and compares it to its actual measured throughput once.
4. **Window Adjustment** : window is adjusted acc to the difference between expected and actual rates using two thresholds, α and β . If the difference is below α , the window increases linearly; if it is above β , it decreases linearly; if it falls between the two, the window stays unchanged.
5. **Controlled Exponential Growth**: To avoid the massive packet losses like that of standard TCP, Vegas only permits exponential window growth every other rtt
6. **Safe Transition**: During alternate rtt the window is fixed to measure actual throughput if it falls below the expected rate by a set threshold, Vegas transitions out of slow start into linear mode

Suitable Scenarios where it works well :

1. **Small Data Transfers:** For smaller file sizes Vegas modified slow start effectively avoids packet losses. The amount of data it needs to retransmit decreases almost linearly.
2. **Networks with Adequate Router Buffers:** When intermediate routers have sufficient buffer space Vegas congestion avoidance function optimally and maintains the correct amount of extra data in transit to utilize available bandwidth
3. **Mixed Background Traffic Environments:** Vegas adapts smoothly to varying levels of network load by making efficient use of available bandwidth. It maintains high performance across all levels of congestion without aggressively stealing bandwidth

Suitable Scenarios it fails

1. **Inaccurate BaseRTT:** Vegas depends heavily on an accurate BaseRTT estimate if this estimate is too small, throughput artificially drops, and if it is too large, the protocol will mistakenly overrun the connection.
2. **Heavy Congestion with Limited Buffers:** When network congestion is severe or intermediate routers have very limited buffer space, Vegas's mechanisms cannot function effectively and it falls back to behaving like standard Reno.
3. **Slow Start in Low Buffer Routers:** During slow start, TCP still sends segments at twice the supported rate; if a bottleneck router lacks buffers Vegas may still lose segments before receiving network feedback to slow down

Hybrid TCP Veno

TCP Veno is an enhanced data transmission algorithm designed to improve internet speeds particularly over wireless networks where random data loss is common. Other algorithms often mistake random wireless errors for congestion causing them to unnecessarily slow down the data transfer. it maintains higher data speeds and makes better use of available bandwidth, all while only requiring updates on the sender's side

Algorithm Details :

1. **Monitors the network state:** It continuously calculates the expected data rate versus the actual data rate to estimate if packets are accumulating in network routers.
2. **Identifies the type of loss:** If a packet drops but the network doesn't look crowded, veno assumes it was a random error like noise, If the network is crowded, it assumes it is a congestion loss.
3. **Gentle speed reduction for random errors:** When it detects a random error, Veno only slightly reduces its sending speed reducing its window size by a factor of $4/5$ rather than drastically slowing down.
4. **Standard speed reduction for real traffic jams:** When a real congestion loss occurs, it acts exactly like standard algorithms and cuts its sending speed in half to help clear the network.
5. **Careful speed increases:** When Veno senses that the network buffer is filling up, it slows down its increasing sending speed, allowing the connection to stay at an optimal speed for longer time.

Suitable Scenario where it works well :

1. **Wireless and cellular networks:** It performs well in environments like Wi-Fi or mobile networks where random packet loss happens frequently due to signal noise or interference.
2. **Shared networks:** It works well when sharing a network with older, standard connections (like TCP Reno) because it uses leftover bandwidth efficiently rather than aggressively stealing speed
3. **Metropolitan Wide Area Networks (WANs):** It successfully improves overall data throughput in city wide networks and heterogeneous networks containing mixed wired and wireless links.

Suitable Scenario where it fails :

1. **Extremely high-loss networks:** If the random data loss rate becomes extremely heavy Venos timing mechanism breaks down and it suffers from constant timeouts performing no better than standard algorithms.
2. **Bursty packet loss:** Veno alone struggles to handle situations where a large group of packets is lost all at once in quick succession though this can be fixed by pairing it with an add on called SACK.
3. **Perfectly clean wired networks:** In low loss, purely wired environments without random errors Venos special features are rarely triggered meaning it provides roughly the same speed as standard algorithms with no major extra benefits.

Loss Based - TCP Westwood

TCP Westwood Overview TCP Westwood is an enhanced version of the standard Transmission Control Protocol (TCP) designed to improve data transfer speeds, particularly over unreliable connections. When it detects a loss, it uses this real-time bandwidth estimate to intelligently reset its sending limits, allowing the connection to recover much faster than traditional TCP methods

How the Algorithm Works

1. **Continuous Monitoring:** The sender continuously tracks how fast acknowledgment messages are coming back from the receiver to figure out how much data is actually getting through.
2. **Filtering the Data:** It applies a mathematical formula (a low-pass filter) to these measurements to smooth out sudden jumps and calculate a stable estimate of the available bandwidth.
3. **Smarter Reductions:** When minor packet losses happen (indicated by receiving 3 dup Acks), the algorithm uses its bandwidth estimate to calculate a new transmission threshold, rather than just automatically cutting its sending window in half.
4. **Handling Major Delays:** If a major connection timeout occurs, it drops its sending window down to 1 to ensure fairness with other network users, but it uses the bandwidth estimate to set a target threshold so it can ramp back up to optimal speeds quickly.
5. **Preventing Errors:** It has built-in rules to carefully count delayed or grouped ACKs, ensuring that sudden bursts of acknowledgments do not trick the system into overestimating the network speed.

Suitable Scenarios where it works well :

1. **Wireless and Unreliable Links:** It excels on networks like Wi-Fi or mobile connections where data packets are often dropped randomly due to a bad signal rather than actual network traffic jams.
2. **Long-Distance, High-Speed Connections:** It performs exceptionally well on "fat pipes" with long delays, such as satellite links, because it avoids the overly cautious speed reductions that cripple traditional TCP.

3. **Networks with Bursty Traffic:** It recovers very quickly in environments where other users are sending unpredictable, sudden bursts of heavy traffic that cause brief, sporadic packet losses.

Suitable Scenarios where it fails

1. **Connections with Very Low Delay:** On networks with very small Round Trip Times (RTT), the algorithm can be too aggressive, leading to inaccurate bandwidth estimates that actually force it to slow down too often.
2. **Sudden Increases in Network Speed:** It is slow to realize when extra bandwidth suddenly becomes available, because its estimation formula relies on the rate of data it is currently sending.
3. **Prolonged Heavy Congestion:** During long, sustained periods of network traffic jams, the algorithm can sometimes overestimate how much bandwidth is actually available.

FINAL COMPARISON AMONG THE ALGORITHMS

Method of Detection :

Westwood and **CUBIC** are **Loss-Based**, relying on packet drops (like 3 duplicate ACKs) to signal congestion. **Vegas** is **Delay-Based** monitoring Round Trip Time to detect slowdowns before a drop occurs. **Veno** is a **Hybrid** approach, utilizing both delay monitoring and packet loss to assess network state.

Network Traffic Management :

Westwood and **CUBIC** are strictly **Reactive**, pushing the network until a packet is dropped, then adjusting speed. **Vegas** is **Proactive**, aiming to prevent congestion by slowing down early based on RTT changes. **Veno** is mostly reactive but uses **proactive** RTT checks to calibrate the severity of its reaction to a loss.

Window Growth Mechanics and RTT Dependence :

CUBIC is **RTT-independent**, growing its window based on a cubic time curve for fairness on long links. **Westwood** and **Veno** are **RTT-dependent**; Westwood tracks the ACK rate to estimate bandwidth, while Veno uses simple linear growth. **Vegas** relies heavily on **RTT measurements** to compare expected vs. actual throughput.

Reaction to Packet Loss :

Upon loss, **Westwood** uses its bandwidth estimate to set a new threshold, avoiding the default halving. **CUBIC** cuts its speed by a smaller factor (20%) for faster recovery. **Veno** is adaptive: it cuts speed by only 20% for random errors but cuts by 50% for confirmed congestion loss. **Vegas** usually avoids loss entirely but defaults to standard recovery methods if forced.

Tabular Data for the comparison is provided below :

Feature	TCP CUBIC	TCP Vegas	TCP Westwood	TCP Veno
Congestion Class	Loss-Based	Delay-Based	Loss-Based	Hybrid (Loss & Delay)
Control Strategy	Reactive (Adjusts after drop)	Proactive (Prevents drops)	Reactive (Adjusts after drop)	Smart Reactive (Calibrated by delay)
Detection Mechanism	Packet loss	RTT increase / delay	Packet loss (3 dup ACKs)	Loss + Network Delay
RTT Dependence	Independent (Grows by elapsed time/cubic function)	Highly Dependent (Relies on BaseRTT estimate)	Dependent (Tracks ACK rate/bandwidth estimate)	Dependent (Uses linear growth)
Rate Increase Method	Cubic function (Curve)	Linear difference calculation	Tracks ACK rate for bandwidth estimate	Refined Additive Increase
Loss Reaction	Cuts speed by 20%	N/A (Attempts to avoid loss)	Sets new threshold using bandwidth estimate (avoids 50% cut)	20% cut for random error; 50% cut for real congestion

Loss Handling	Poor (Treats errors as congestion)	Poor (May slow down early)	Excellent (Avoids aggressive speed reduction)	Excellent (Distinguishes errors)
Optimal Use Case	Fast, Long-Distance Networks (High BDP)	Short Transfers, Adequate Router Buffers	Wireless/Unreliable Links, Long-Delay Connections	Wireless and Mobile Networks
Major Limitation	Slow convergence speed due to small decrease factor (0.2)	Fails with inaccurate BaseRTT estimate	Can be too aggressive on very low-delay networks	Fails in extremely high-loss networks

Inferences :

- Vegas is the only algorithm that tries to prevent packet loss before it happens by watching network delays. CUBIC, Westwood, and Veno wait until a packet is actually dropped before fixing the problem.
- Older TCP assumes any dropped packet is a traffic jam. Veno and Westwood are specifically designed to know the difference between a real traffic jam and a random Wi-Fi/cellular signal drop.
- CUBIC dominates fast, long-distance internet because its speed relies on a clock (elapsed time) rather than how long a signal takes(RTT).
- When a drop does happen, CUBIC, Westwood, and Veno all reject the earlier rule of reducing speed by 50%. Instead, they use mathematical or bandwidth tracking (dropping speed by roughly 20%).

- Westwood acts like a constant calculator, using continuous math (filters) to estimate exact bandwidth. CUBIC avoids the heavy estimation and just uses a simple clock (elapsed time) to pace its growth.
- if a network suffers from sudden bursts of lost packets Westwood thrives and recovers instantly. Veno, chokes on sudden bursts of drops unless we apply extra software patches to it.
- Vegas breaks if its initial speed guess is wrong. CUBIC is more likely to grab new bandwidth if the network suddenly clears up and Veno/Westwood lose their advantage if a network is either perfectly clean or completely falling apart

PART 2

Shell Script Code :

```
$ run.sh

1  #!/bin/bash
2  mkdir -p results
3
4  declare -a tcpvar=("TcpNewReno" "TcpHighSpeed" "TcpVeno" "TcpVegas")
5  declare -a bws=("20Mbps" "50Mbps" "100Mbps")
6  declare -a lcs=("20ms" "50ms" "100ms")
7
8  for tcp in "${tcpvar[@]}; do
9
10     name=$(echo "$tcp" | sed 's/Tcp//' | tr '[:upper:]' '[:lower:]')
11
12     echo "testing $tcp"
13     echo " "
14
15
16     echo "[Base setup: 5Mbps, 2ms]"
17     ./ns3 run "as7_que2 --tcpVariant=$tcp --linkBw=5Mbps --linkDelay=2ms" > ns3_output.log 2>&1
18
19     # saveig plots acc to the name defines inthe ques
20     mv seventh.cwnd results/${name}_cwnd.txt
21     mv seventh_rtt.txt results/${name}_rtt.txt
22     mv seventh-sssthresh.txt results/${name}_sssthresh.txt
23     mv seventh-file-0-0.pcap results/${name}_node0.pcap
24     mv seventh-file-1-0.pcap results/${name}_node1.pcap
25
26     tput=$(capinfos -i results/${name}_node1.pcap | grep "Data bit rate" | awk -F: '{print $2}' | xargs)
27     echo "    Avg throughput: $tput"
28
29     drops=$(awk '3 < $2 {c++} END {print c+0}' results/${name}_cwnd.txt)
30     echo "    Cwnd dropped $drops times "
31
32     echo -e "\n testing changing Bandwidth  (delay fixed at 2ms)"
33     for bw in "${bws[@]}; do
34         ./ns3 run "as7_que2 --tcpVariant=$tcp --linkBw=$bw --linkDelay=2ms" >> ns3_output.log 2>&1
35
36         tmp_pcap="results/${name}_${bw}_2ms_node1.pcap"
37         mv seventh-file-1-0.pcap $tmp_pcap
38         rm -f seventh-file-0-0.pcap seventh.cwnd seventh_rtt.txt seventh-sssthresh.txt
39
40         b_tput=$(capinfos -i $tmp_pcap | grep "Data bit rate" | awk -F: '{print $2}' | xargs)
41         echo "    BW: $bw , Throughput: $b_tput"
42     done
43
44     echo -e "\n testing changing latency (BW fixed at 5Mbps)"
45     for del in "${lcs[@]}; do
46         ./ns3 run "as7_que2 --tcpVariant=$tcp --linkBw=5Mbps --linkDelay=$del" >> ns3_output.log 2>&1
47
48         tmp_pcap="results/${name}_5Mbps_${del}_node1.pcap"
49         mv seventh-file-1-0.pcap $tmp_pcap
50         rm -f seventh-file-0-0.pcap seventh.cwnd seventh_rtt.txt seventh-sssthresh.txt
51
52         d_tput=$(capinfos -i $tmp_pcap | grep "Data bit rate" | awk -F: '{print $2}' | xargs)
53         echo "    Delay: $del , Throughput: $d_tput"
54     done
55     echo " "
56 done
```

Explanation :

We first make arrays for the 4 different tcpvariants , bandwidth, latency then we loop over the tcpvariants and for each var we run the as7_que2.cc code, after which we evaluate the cwnd change , after which we loop over the bandwidth keeping latency fixed to above the change in

avg throughput change and similarly for the latency change effect, we also create the results directory where the required pcaps , gnuplots codes images, .txt for the gnuplot code data is stored, the output of the change in cwnd , bandwidth change , latency change is printed on the terminal itself.

Question 1

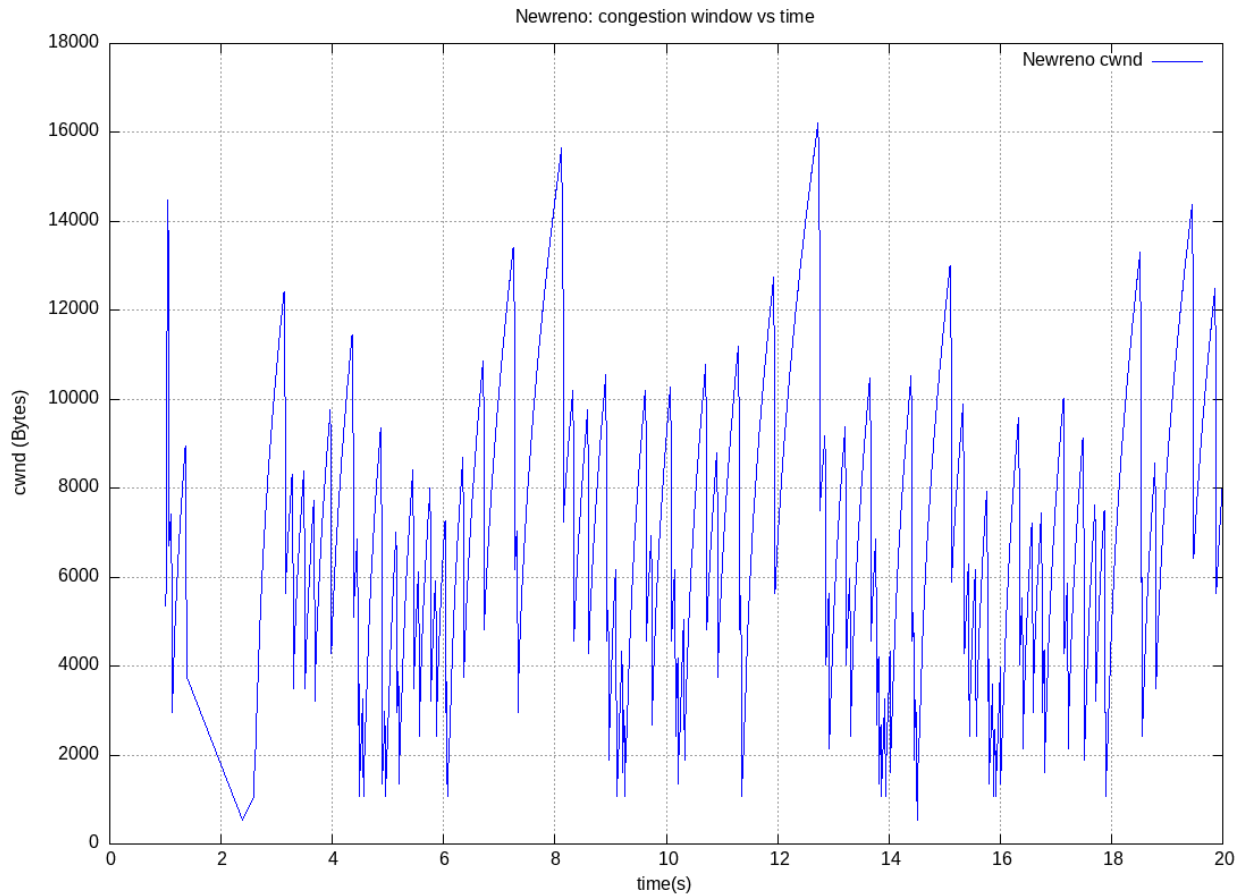
Gnuplot Code :

```
❏ cwnd.gnuplot
1  set terminal png size 1100,800
2  set xlabel 'time(s)'
3  set ylabel 'cwnd (Bytes)'
4  set grid
5
6  set output 'results/newreno_cwnd.png'
7  set title 'Newreno: congestion window vs time'
8  plot 'results/newreno_cwnd.txt' using 1:3 with lines linecolor rgb "blue" title 'Newreno cwnd'
9
10 set output 'results/highspeed_cwnd.png'
11 set title 'Highspeed: congestion window vs time'
12 plot 'results/highspeed_cwnd.txt' using 1:3 with lines linecolor rgb "red" title 'Highspeed cwnd'
13
14 set output 'results/veno_cwnd.png'
15 set title 'Veno: congestion window vs time'
16 plot 'results/veno_cwnd.txt' using 1:3 with lines linecolor rgb "dark-green" title 'Veno cwnd'
17
18 set output 'results/vegas_cwnd.png'
19 set title 'Vegas: congestion window vs time'
20 plot 'results/vegas_cwnd.txt' using 1:3 with lines linecolor rgb "purple" title 'Vegas cwnd'
```

COMMAND USED : gnuplot cwnd.gnuplot

This produces the 4 different cwnd vs time graphs for the 4 TCP congestion control algorithms mentioned in the question.

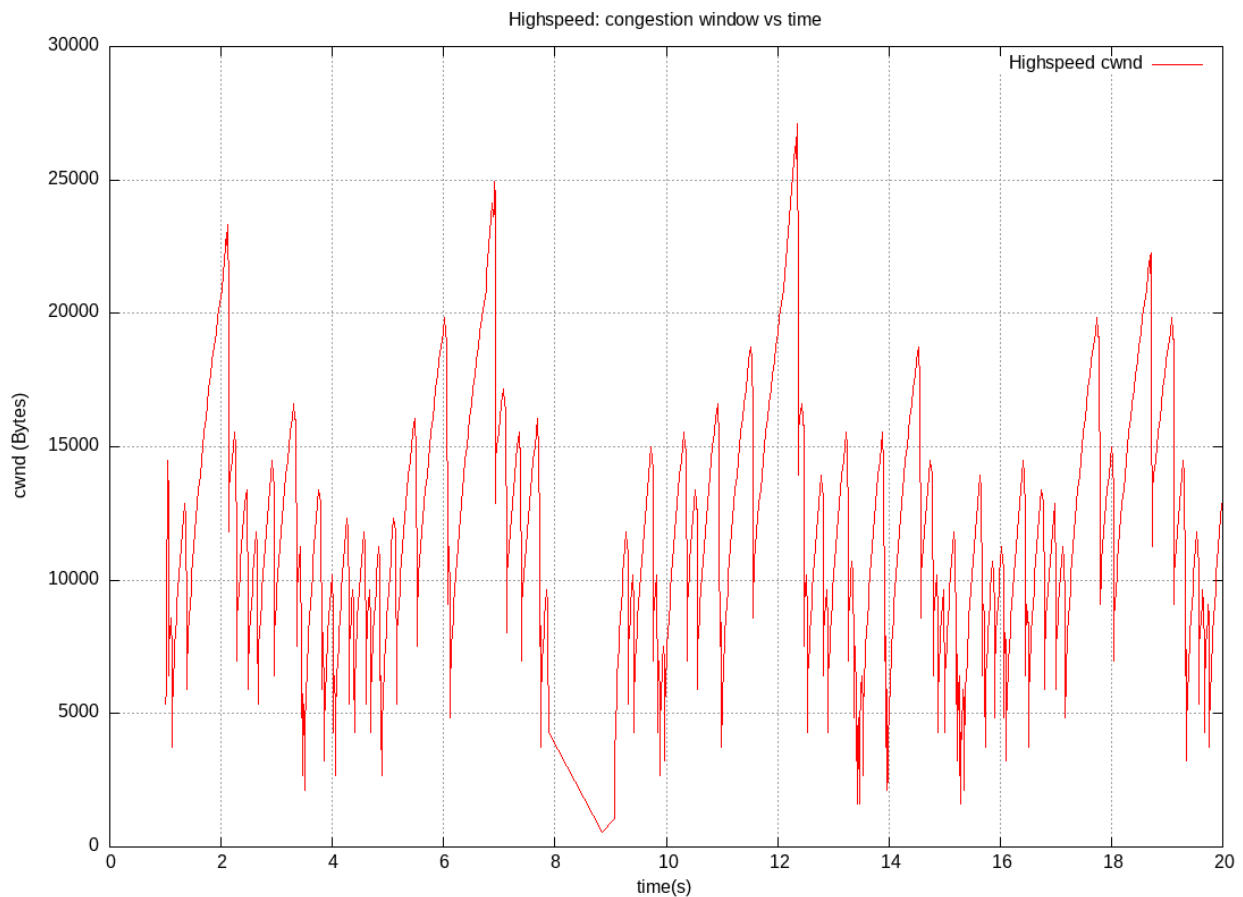
1. Newreno



Observation :

1. Initially at 1 second graph goes up to 14500 bytes and then drops sharply (first massive spike is **slow start phase** where the TCP double the congestion window window)
2. steady diagonal line are **congestion avoided** where the algorithm increases by The Window by **1 MSS** until packet drops
3. After the first few seconds the graph settles into a **repeating pattern** i.e going up diagonally and then falling **roughly by half**. (The half drop indicate **fast recovery** where the algorithm detects duplicate ack and cut the window size by half)

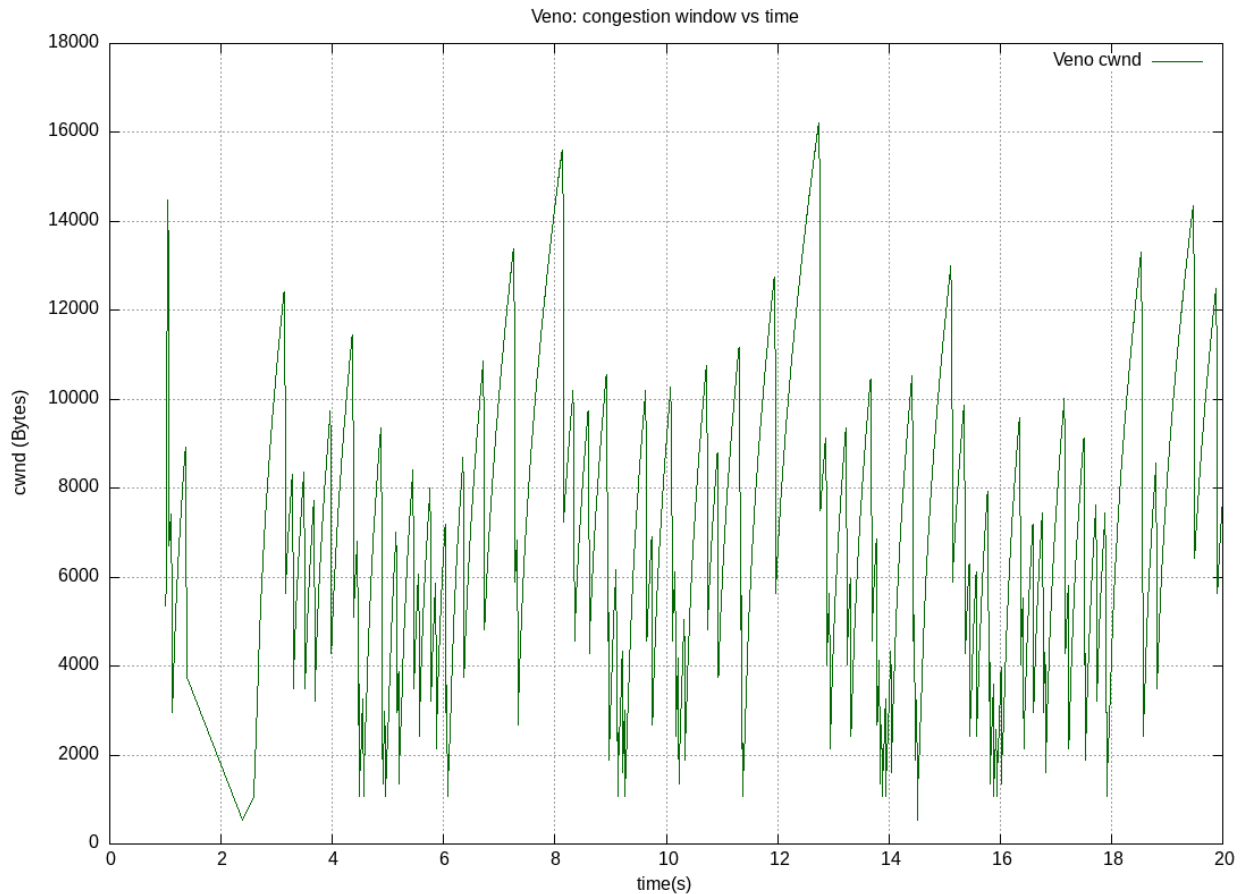
2. Highspeed



Observation :

1. cwnd reaches peak between 20000 to around 27000 almost double of what newreno achieved.
2. Initial sharp peak after 1 sec is the **slow start phase**.
3. standard TCP grows too slowly on modern high bandwidth links so highspeed modifies the **Congestion Avoidance** so that the larger the window gets, the faster it is allowed to keep growing as seen by the steep accelerating upward slope.
4. The drop around 9th sec can be because of **Retransmission Timeout**, meaning no Acks were received and was forced to reset to 1 MSS.

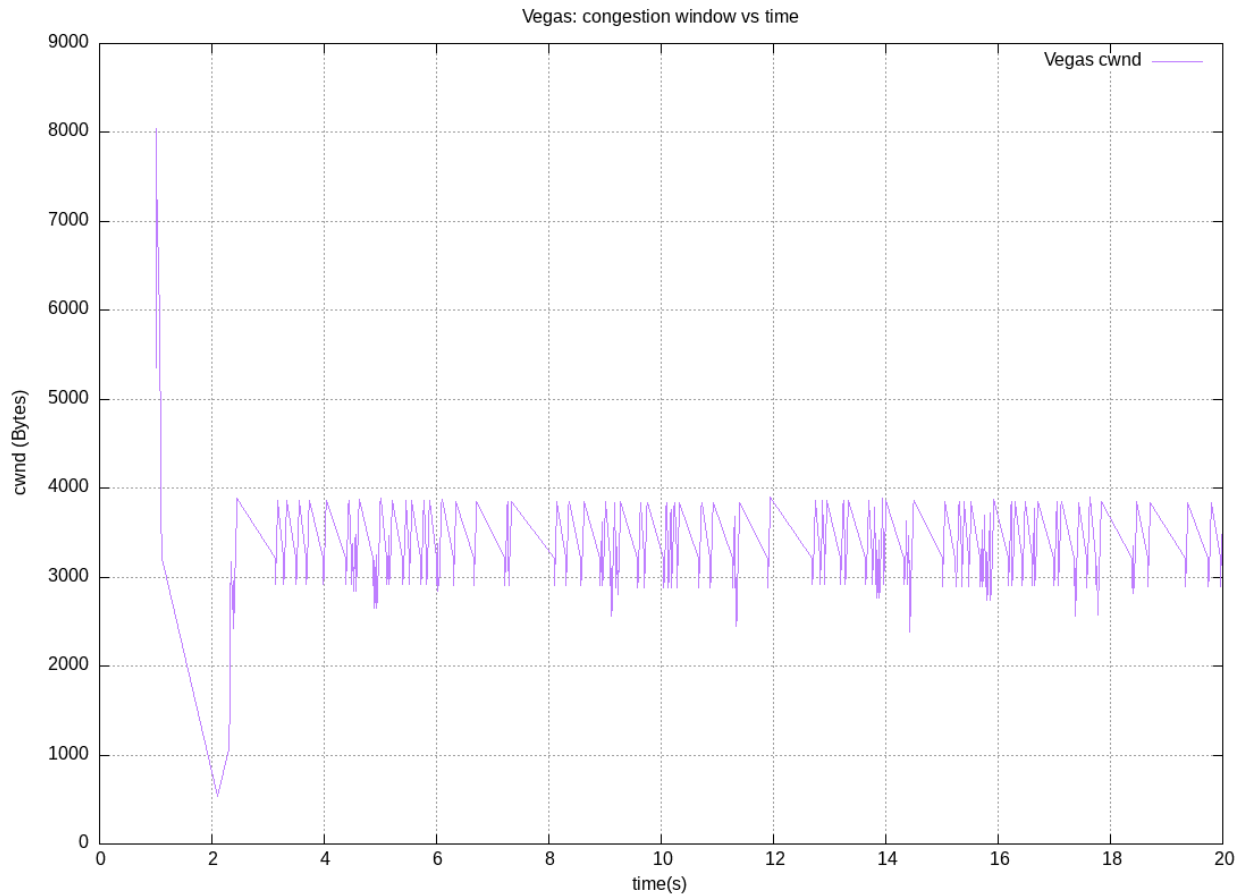
3. Veno



Observation :

1. It peaks around 14500 bytes , and has a sawtooth kind of pattern which is the **slow start phase**.
2. There is a severe drop right after the initial Slow Start spike (around 2 seconds), down to near zero (once that packet drops veno cuts its window down to recover via **multiplicative decrease**).
3. The shaky nature of the graph reflects Veno making constant, small adjustments during **congestion avoidance** as it evaluates the state of the connection.

4. Vegas



Observation :

1. After an initial spike and drop in the first 3 seconds which is the **slow start phase**, the large pattern similar to seen in other completely vanishes
2. the window size stabilizes perfectly, bouncing rapidly inside a very narrow band between around 3,000 and 4,000 bytes which is a result of **Vegas's congestion avoidance** which works on the relation between **expected RTT and actual RTT**.
3. it enters that **flatline phase**, there are absolutely no severe vertical drops down to zero as result it tries to **ensure retransmission timeouts are minimum**.

Question 2.

COMMAND USED : ./run.sh

TcpVariant	Avg Throughput (kbps)
Newreno	4743
Highspeed	4965
Veno	4738
Vegas	4763

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ ./run.sh
testing TcpNewReno

[Base setup: 5Mbps, 2ms]
  Avg throughput: 4,743 kbps
  Cwnd dropped 763 times

testing changing Bandwidth (delay fixed at 2ms)
  BW: 20Mbps , Throughput: 9,803 kbps
  BW: 50Mbps , Throughput: 10 Mbps
  BW: 100Mbps , Throughput: 10 Mbps

testing changing latency (BW fixed at 5Mbps)
  Delay: 20ms , Throughput: 1,080 kbps
  Delay: 50ms , Throughput: 548 kbps
  Delay: 100ms , Throughput: 277 kbps

testing TcpHighSpeed

[Base setup: 5Mbps, 2ms]
  Avg throughput: 4,965 kbps
  Cwnd dropped 1631 times

testing changing Bandwidth (delay fixed at 2ms)
  BW: 20Mbps , Throughput: 15 Mbps
  BW: 50Mbps , Throughput: 19 Mbps
  BW: 100Mbps , Throughput: 19 Mbps

testing changing latency (BW fixed at 5Mbps)
  Delay: 20ms , Throughput: 2,181 kbps
  Delay: 50ms , Throughput: 902 kbps
  Delay: 100ms , Throughput: 504 kbps

testing TcpVeno

[Base setup: 5Mbps, 2ms]
  Avg throughput: 4,738 kbps
  Cwnd dropped 763 times

testing changing Bandwidth (delay fixed at 2ms)
  BW: 20Mbps , Throughput: 10 Mbps
  BW: 50Mbps , Throughput: 13 Mbps
  BW: 100Mbps , Throughput: 14 Mbps

testing changing latency (BW fixed at 5Mbps)
  Delay: 20ms , Throughput: 1,696 kbps
  Delay: 50ms , Throughput: 733 kbps
  Delay: 100ms , Throughput: 392 kbps
```

```

testing TcpVegas

[Base setup: 5Mbps, 2ms]
  Avg throughput: 4,763 kbps
  Cwnd dropped 165 times

testing changing Bandwidth (delay fixed at 2ms)
  BW: 20Mbps , Throughput: 7,951 kbps
  BW: 50Mbps , Throughput: 15 Mbps
  BW: 100Mbps , Throughput: 29 Mbps

testing changing latency (BW fixed at 5Mbps)
  Delay: 20ms , Throughput: 1,891 kbps
  Delay: 50ms , Throughput: 1,510 kbps
  Delay: 100ms , Throughput: 771 kbps

Done running all tests. Files are in /results.

```

TCP **Highspeed** dominates others in terms of avg throughput so it performs best among the given 4 algorithms.

Question 3.

COMMAND USED : ./run.sh

TcpVariant	cwnd reduce count
Newreno	763
Highspeed	1631
Veno	763
Vegas	165

Reasons for cwnd reduced in Newreno :

1. simulation utilizes an error model that randomly discards data units to mimic a lossy, congested network environment
2. Once the sender catches three of these duplicate signals, it assumes severe bottlenecking and triggers a fast recovery state.

3. the algorithm aggressively reacts by instantly halving its transmission window resulting in frequent sharp downward performance spikes.

Reasons for cwnd reduced in Highspeed :

1. Because this variant is purpose-built for massive bandwidth links it evaluates its current window capacity before reacting to those error signals
2. Instead of reducing the window by fifty percent it applies a formula to scale back the transmission rate by a much smaller fraction.
3. aggressive growth keeps overall speeds maximized but constantly pushes the link past its limit, leading to the highest count of transmission failures

Reasons for cwnd reduced in Veno :

1. this protocol constantly tracks network delays alongside standard duplicate acknowledgment monitoring.
2. When data goes triggered via 3 duplicate acks , it uses rtt calculations to estimate whether the loss was just a random wireless interference or router congestion
3. its underlying growth curve mirrors newreno causing it to hit the simulation's error ceiling at the exact same frequency

Reasons for cwnd reduced in Vegas :

1. this delay based algorithm reacts to increasing rtt rather than waiting for forced data drops.
2. It continuously measures the variance between expected throughput and actual delivery speed to monitor router queue capacity in real time.
3. If the system detects the queue filling up, it steps in to change the transmission rate before any data is actually discarded
4. preventative adjustment strategy creates a stable connection with fewer drastic window reductions

Question 4

COMMAND USED : ./run.sh

Effect of Changing Bandwidth (Latency fix at 2 ms)

TcpVariant	BW: 20 Mbps (Throughput in kbps)	BW: 50 Mbps (Throughput in kbps)	BW: 100 Mbps (Throughput in kbps)
Newreno	9,803	10,000	10,000
Highspeed	15,000	19,000	19,000
Veno	10,000	13,000	14,000
Vegas	7,951	15,000	29,000

Effect : Increasing bandwidth generally increases throughput across all TCP variants, but the utilization efficiency varies significantly depending on the specific congestion control algorithm's window scaling mechanism as a result the vegas variant of tcp achieved the highest bandwidth in 100Mbps.

Explanation : Higher bandwidth expands channel capacity as result the tcp congestion algorithms scale much higher before hitting router queue limits and dropping packets.

Effect of Changing Latency (BW fixed at 5Mbps)

TCP Variant	Delay: 20 ms (Throughput in kbps)	Delay: 50 ms (Throughput in kbps)	Delay: 100 ms (Throughput in kbps)
Newreno	1,080	548	277
Highspeed	2,181	902	504
Veno	1,696	733	392
Vegas	1,891	1,510	771

Effect : Increasing network latency reduces the throughput of all TCP variants because higher delays inflate the Round Trip Time and slow down window growth.

Explanation : TCP relies on Acknowledgements (ACKs) to increase its transmission rate. Higher latency delays these ACKs, which severely slows down the rate at which the congestion window can grow during the Slow Start and Congestion Avoidance phases

Question 5

Increasing the default MTU (Maximum Transmission Unit) generally improves average throughput by sending more payload data per TCP/IP header, which reduces network overhead. Furthermore, because the TCP congestion window grows by segments (MSS) rather than raw bytes, a larger MTU allows the actual byte transmission rate to ramp up much faster. Conversely, larger MTU sizes can actively harm performance in lossy environments, which we see simulated by our RateErrorModel. Because losing a single large packet requires retransmitting a massive amount of wasted data, scaling down to a smaller MTU provides much better reliability and efficiency when network stability is low.

Question 6

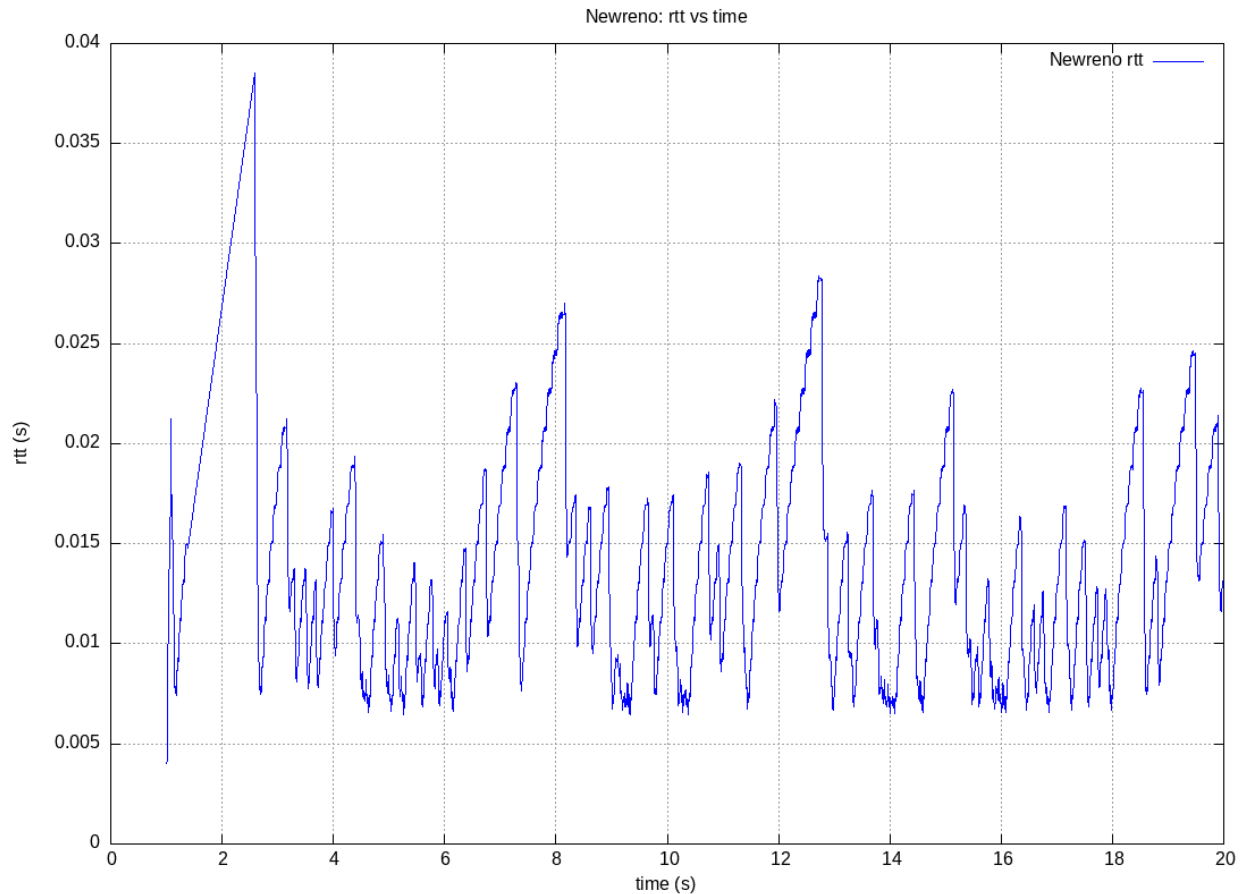
Gnuplot code :

```
rtt.gnuplot
1  set terminal png size 1100,800
2  set xlabel 'time (s)'
3  set ylabel 'rtt (s)'
4  set grid
5
6  set output 'results/newreno_rtt.png'
7  set title 'Newreno: rtt vs time'
8  plot 'results/newreno_rtt.txt' using 1:3 with lines linecolor rgb "blue" title 'Newreno rtt'
9
10 set output 'results/highspeed_rtt.png'
11 set title 'Highspeed: rtt vs time'
12 plot 'results/highspeed_rtt.txt' using 1:3 with lines linecolor rgb "red" title 'Highspeed rtt'
13
14 set output 'results/veno_rtt.png'
15 set title 'Veno: rtt vs time'
16 plot 'results/veno_rtt.txt' using 1:3 with lines linecolor rgb "dark-green" title 'Veno rtt'
17
18 set output 'results/vegas_rtt.png'
19 set title 'Vegas: rtt vs time'
20 plot 'results/vegas_rtt.txt' using 1:3 with lines linecolor rgb "purple" title 'Vegas rtt'
```

COMMAND USED : gnuplot rtt.gnuplot

This produces 4 different rtt vs time graph for the 4 TCP congestion control algorithms mentioned in the question

1. Newreno



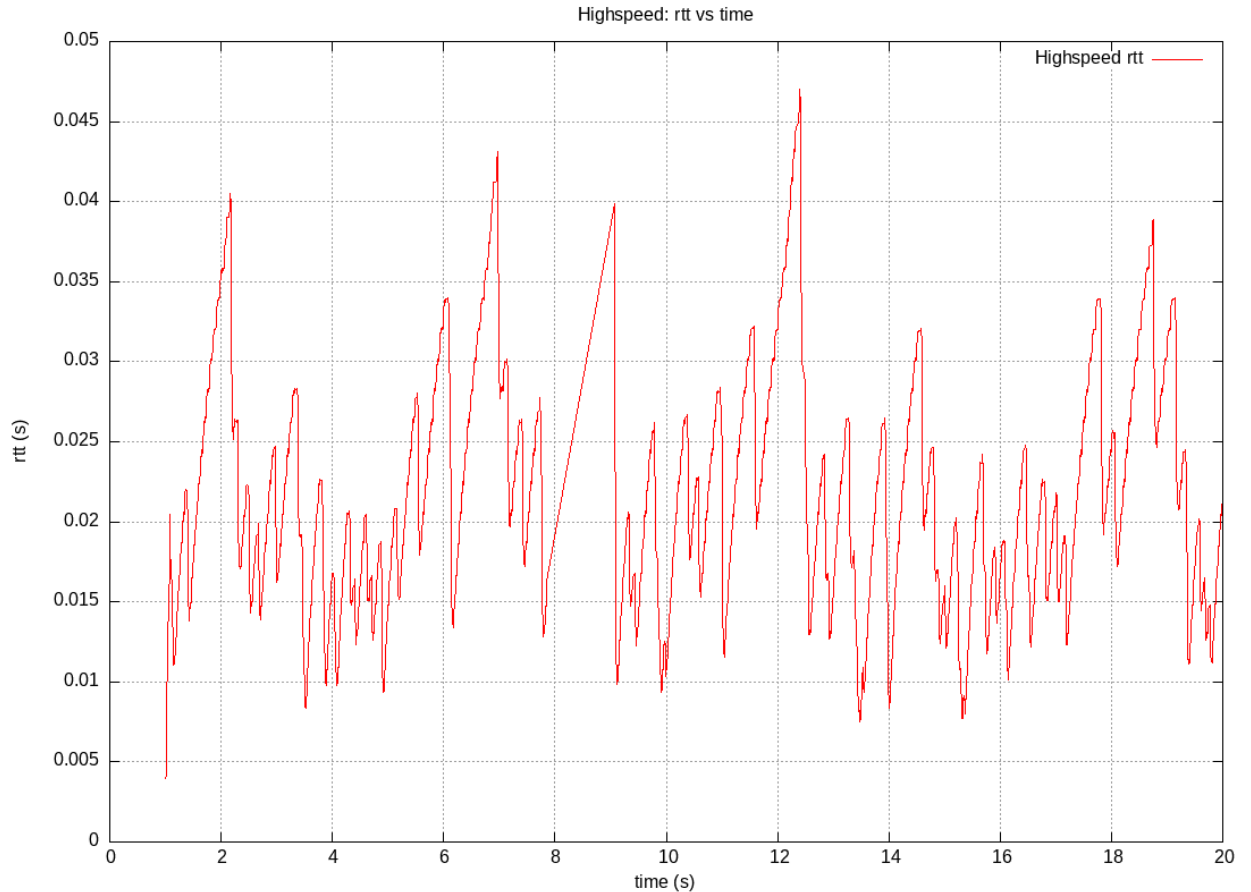
Observation

1. RTT follows a distinct sawtooth pattern, repeatedly climbing to peaks around 25ms before sharply dropping.
2. After every peak, the RTT instantly plunges back down to a baseline of roughly 5ms to 7ms.

Inference

1. The rising RTT directly reflects increasing queueing delay as Node 0's packets fill up the router's buffer.
2. The sharp drops happen when packet loss forces the sender to halve its window, instantly giving the router time to drain its congested queue.

2. Highspeed



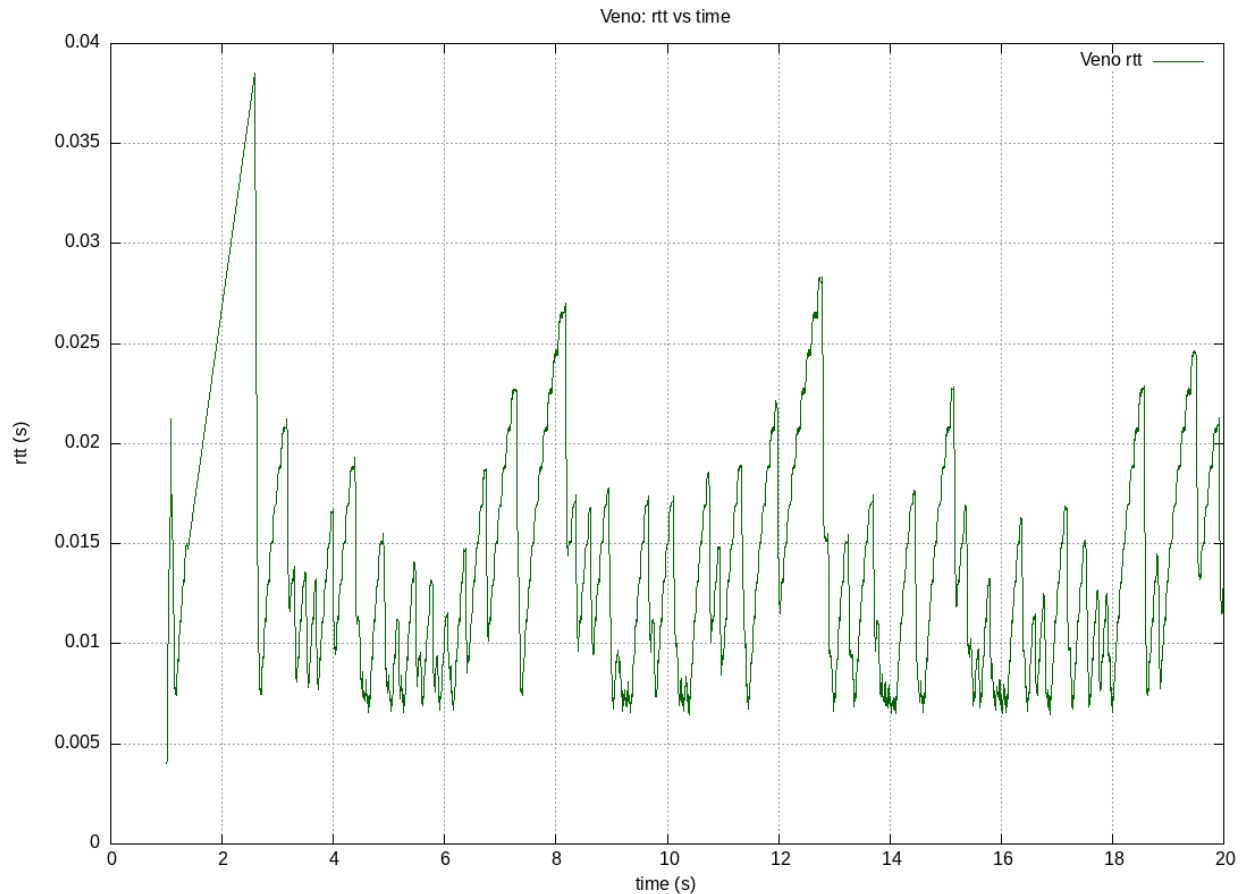
Observation

1. The RTT is highly volatile, frequently hitting much higher peaks up to 45ms compared to NewReno.
2. The upward climbs to peak RTT are much steeper and occur more frequently.

Inference

1. HighSpeed's aggressive window growth pumps data into the network much faster, resulting in higher and more rapid queueing delays.
2. The larger peaks show that HighSpeed constantly pushes the router's buffer capacity to its absolute physical limit before a drop occurs.

3. Veno



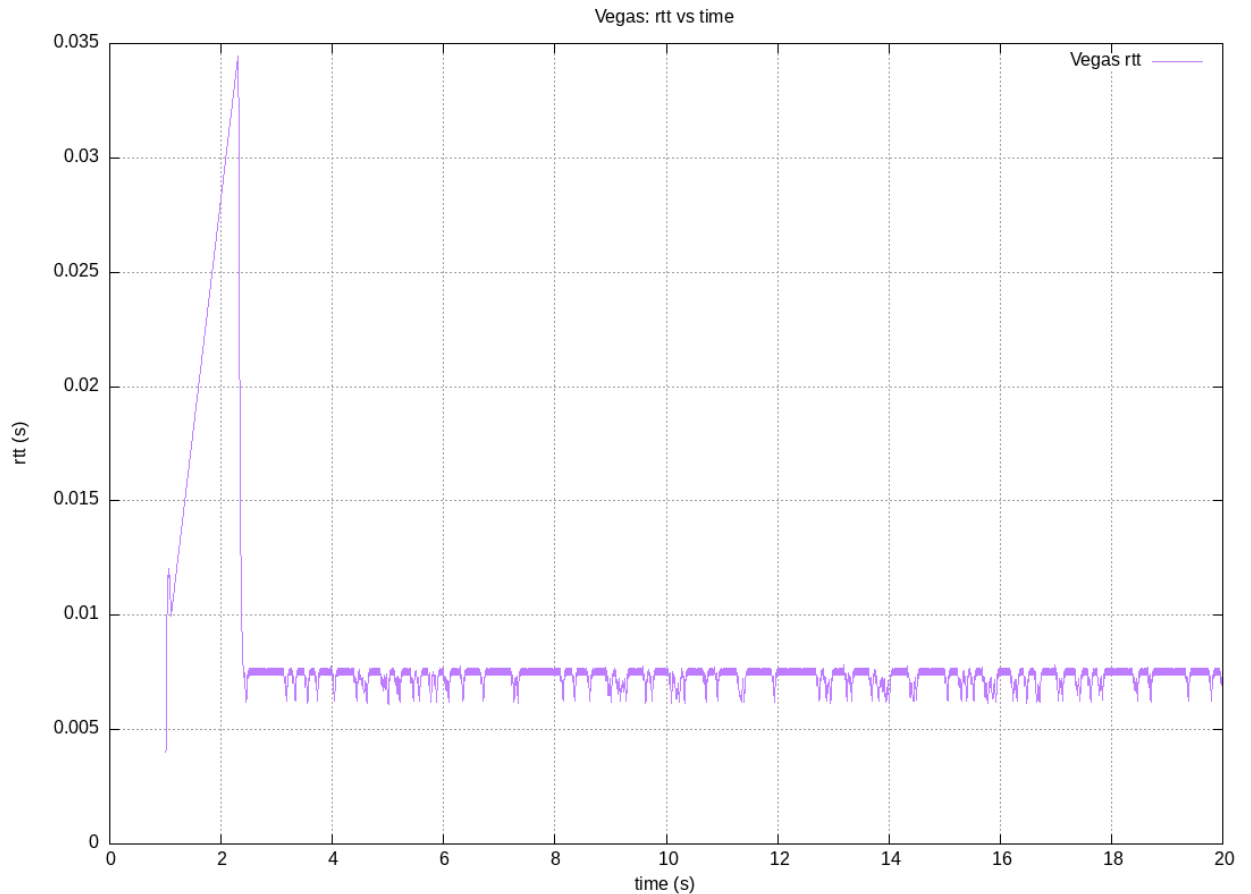
Observation

1. The graph behaves almost identically to NewReno, peaking around 25ms to 28ms during regular cycles.
2. There are slight fluctuations within the lower bounds of the graph instead of perfectly clean drops.

Inference

1. Because Veno shares Newreno's foundational loss-based growth curve, it causes identical buffer-filling behaviors and delays.
2. The minor flickers reflect Veno constantly monitoring the queueing delay to try and differentiate between network congestion and random wireless errors.

4. Vegas



Observation

1. After a single initial spike, the rtt completely flatlines stabilizing in a tight band.
2. There are absolutely no major spikes, or crashes after the 3-sec.

Inferences

1. Because Vegas is delay based, it reduces its window size before the router queue overflows preventing queuing delay spikes.
2. The flatline proves Vegas successfully found the optimal sending rate, keeping a stable number of packets in the queue without triggering any drops.